

Integrating Software Product Line Engineering and Agile Development

Kannan Mohan, *Baruch College*

Balasubramaniam Ramesh, *Georgia State University*

Vijayan Sugumaran, *Oakland University*

Top management, project managers, and developers in software development organizations can implement successful practices to integrate software product line engineering and agile development methods.

Can software product line engineering (SPLE) be agile? This article presents successful practices that integrate SPLE and agile methods. SPLE has gained considerable attention in research and practice by addressing diverse market needs while controlling costs through planned reuse.¹ It typically follows a mature process involving domain and application engineering. Although SPLE can reduce cycle time in developing product variants, it involves significant up-front commitment in developing a flexible product platform.

In a dynamic market environment with stringent time-to-market constraints, software development organizations need agility in handling changes. Agile development methods are increasingly used for this purpose. Agile methods such as extreme programming (XP) promote iterative development, frequent interaction with the customer, small and frequent releases, and rigorously tested code. The most important characteristic of agile development is its adaptive, responsive approach to requirements changes.²

Although much of the literature and industry practices have focused on applying either SPLE or agile methods in isolation, few studies have examined whether these two seemingly contradictory methods can be integrated. Most SPLE practices relate to a product family's long-term strategic objectives, whereas agile practices focus on short-term tactical objectives of developing a

single product.³ Integrating both practices balances the ability to quickly handle changes and maximize product variety.

SPLE and Agile Methods

A software product line is a set of software-intensive systems sharing a common, managed set of features,¹ developed from reusable core assets and incorporating variations to derive product variants. This involves identifying commonality and variability in the product family and implementing shared artifacts while preserving the ability to implement required variability.¹ Software development organizations that recognize market opportunities for products that share a significant number of common elements, but that also exhibit variations, can reap significant economic benefits with SPLE. For example, Cummins builds a variety of software systems for its

diesel engines using SPLE to reduce cycle time. Although SPLE facilitates commonality and planned variation across different products, the process requires adaptation to incorporate unplanned variations in the product line.

Although SPLE focuses on developing a robust, flexible product platform that can be reused to quickly deliver product variants, it goes beyond traditional code reuse through initial planning and development of the product platform, which involves significant up-front commitment and a disciplined approach. Given the dynamic environmental conditions that demand not only product variety but also quick turnaround, firms must employ product development methods that balance flexibility and discipline.

Agile methods emphasize improvisation over conventional development approaches. These methods focus on quick development in an uncertain, ill-understood environment in which requirements rapidly evolve. These methods also view people, rather than formal documentation, as a project's most important element.

Conflicts between SPLE and Agile Development

The tension between agility and discipline poses interesting challenges for SPLE. At first glance, these approaches appear diametrically opposed. Recognizing the need for agility, researchers have attempted to refine, extend, and conflate disciplined processes to make them more agile.⁴ Past research has highlighted the links and conflicts between product line and agile development.^{3,5} Whereas SPLE focuses on upfront planning and predicting variability in requirements, agile development advocates just enough planning. Current research and practice consider SPLE as involving mature processes with early architecture design and detailed commonality and variability analyses. On the contrary, agile development focuses on just the requirements that must be immediately implemented without much focus on anticipating future requirements. Whereas SPLE emphasizes product platform adaptability to accommodate future variabilities, agile development refrains from developing futuristic designs. Typically, owing to the complexity of SPLE projects, formalized documentation is considered an effective means of communication, whereas agile methods emphasize face-to-face and informal communication.

However, the overall objectives of SPLE and agile methods overlap considerably. Both aim to handle changes in customer requirements. Whereas SPLE uses a planned, platform-based approach,

agile methods use an adaptable, lean process to enable quick direction changes. Integrating the two approaches has gained significant interest in theory and practice. A recent issue of the *Journal of Systems and Software* focuses on integrating these two seemingly conflicting methods and highlights the relevance of various agile principles for SPLE planning, suggesting a planned approach for domain engineering and an agile approach to application engineering.⁶ Integrating SPLE and agile methods can effectively address time-to-market constraints, evolving requirements, and the need for product variety. We draw from the theory of complex adaptive systems (CAS) to develop a foundation for integrating the practices.

Complex Adaptive Systems and Systems Development Process

Past research has used complexity theory to understand systems composed of interacting agents that result in emergent behavior patterns.⁷ Systems that demonstrate coherence in the face of change have been characterized as CAS. Whereas past research has extensively used CAS to study a variety of domains, its application in examining software development methods is relatively nascent but gaining significant attention. For instance, Vidgen and Wang have used CAS principles of coevolving systems to develop theoretical grounding for agile development methods.⁸ Past research has also used CAS to develop underlying principles that help us understand how agile development methods adapt to dynamic environmental conditions.^{9,10}

Hind Benbya and Bill McKelvey described information systems development using seven principles drawn from CAS.¹¹ We use these principles to explain the integrated agile and SPLE practices we identified in a published case study.¹²

Adaptive Tension

Tensions from environmental conditions result in the creation of new kinds of ordered structures to enable adaptation. Such tension can result from a lack of market share or being uncompetitive in innovation, product design, costs, production, or IS design or use. In software development, adaptive tension results from conflicts among the needs of stakeholders whose views shift and evolve as the development process progresses. As the stakeholders' understanding and articulation of their needs evolve, adaptive tension increases. Addressing adaptive tension requires focusing on aligning the processes with evolving organizational needs, which in turn depends on evolving stakeholder needs.

Integrating SPLE and agile methods can effectively address time-to-market constraints, evolving requirements, and the need for product variety.

They developed the initial set of systems and then refactored them to define commonality and variability.

Requisite Complexity

Adaptive order creation occurs if internal complexity exceeds external complexity. Software development organizations handle environmental changes through continuous adaptations. When they can't build sufficient internal complexity up front, they must continuously adapt organizational resources and processes to handle environmental changes.

Change Rate

The rate of variation constrains adaptation (depending on the domain in consideration, this variation could be in knowledge, innovation, learning, and so on). In software development, when issues or needs surface that either are unpredictable up front or need immediate reaction, quick and short cycles of development are appropriate. The ability to adapt internal processes to match environmental variations helps reduce the gap between user expectations and the implemented system.

Modular Design

Complex systems are hierarchical and consist of decomposable subunits that evolve faster toward stable configurations. In systems development, development teams can effectively handle major changes in stakeholder needs with decomposable processes and a system designed with loosely coupled elements (which have limited interdependencies and interact in a well-defined manner).

Positive Feedback

Insignificant events among agents or modules can result in significant order creation. Coevolution is considered a positive feedback process that proceeds toward complex structures. Risk reduction is achieved through positive feedback along the development spirals, prototyping, and defining requirements in an evolutionary fashion rather than attempting to completely define stakeholders' needs up front.

Causal Intricacy

Complex systems adapt by coping with multiple causes (bottom-up, top-down, horizontal, diagonal, and intermittent). Effective adaptation of systems development processes depends on understanding the complexity of links across several critical factors. For example, relationships between technological and organizational factors could be multidirectional and nonlinear. These relationships can dynamically evolve over time.

Coordination Rhythms

The duality of top-down and bottom-up autonomy

creates tensions among agents in a complex system. Recognizing dualities in structures and processes in systems development becomes essential in managing oscillations across different types of development approaches. Alignment among organizational, technological, and strategic perspectives is also necessary.

Integrating Agile and Product-Family Development

Salion, an enterprise software development organization, developed software systems to support revenue acquisition. Its clients were suppliers who developed and marketed complex products by submitting proposals addressing market needs. Salion recognized early on that quick delivery of customizable products was critical for success in its domain. However, it also recognized that managing product variety by developing individual products in separate teams would be overwhelming and make it difficult to quickly deliver customized products. So, Salion realized the need to blend SPLE and agile practices.

Salion used a reactive SPLE approach and incorporated agile principles to adapt to a dynamic market. The organization's reactive approach differed from a more common proactive SPLE approach. They developed the initial set of systems and then refactored them to define commonality and variability, instead of using considerable up-front planning to develop core-assets for the product line.¹²

The Software Product Line Conference acknowledged the excellence of these integrated practices by inducting Salion into the Software Product Line Hall of Fame in 2004. Salion's reactive SPLE approach and using off-the-shelf product line engineering technology helped it "reap the benefits of software product lines while remaining agile in the face of turbulence in [its] application domain" (<http://splc.net/fame/salion.html>). On the basis of our secondary data analysis from a Salion case study using the methods described in the sidebar "How We Conducted the Study," we identify the elements that involve integrating agile and SPLE practices, link the process to contingencies, and use CAS principles to provide a foundation for the integrated practices.

Salion needed to address its domain requirements and agility to respond to customer needs. Contingencies such as the project's nature, domain, market characteristics, and volatility and variety in requirements also affected the development and practice of the integrated process. Table 1 summarizes the link between relevant CAS principles, these principles' implications for process adapta-

tion, and practices that are part of the integrated process.

The seven CAS principles related to the integrated process are interdependent. They're effective in coevolutionary adaptations when applied in concert rather than in isolation.¹¹ This also suggests that we can expect considerable overlap in the links between integrated practices and CAS principles. However, Table 1 identifies only the most relevant links.

Factors That Shape the Integration of SPLE and Agile Methods

Environmental factors provide the conditions that shape the development of integrated processes. For product line development organizations, the market's nature dictates the appropriateness of practices used in developing a set of products. Three environmental factors shaped Salion's integrated practices.

Extent of Variation Required in the Product Portfolio

Depending on the diversity of product requirements in the market, product line organizations must adapt their processes to handle commonality and variability. For example, Salion faced market conditions that required numerous variabilities in its product portfolio. Also, differences in the predictability of variations impacted the development approach.

Time Constraints

Stringent time constraints in delivering customized products dictate the selection of an appropriate approach. Salion's customers required quick delivery of customized products.

Capability Expectations

Salion had to define the capability to produce a product portfolio with adequate functionality before establishing the required process and acquiring the necessary resources and infrastructure.

CAS Principles and the Integrated Process

We link the integrated processes Salion followed to each of the seven CAS principles.

Adaptive Tension

Three integrated practices that Salion implemented are centered on requirements and customer management. They're related to adaptive tension because they focus on addressing environmental conditions by attempting to align processes with evolving organizational needs.

How We Conducted the Study

A published case study on Salion, an enterprise software development organization that develops software systems to support revenue acquisition, served as the primary source of data for our research.^{1,2} The use of such secondary analysis of data is well accepted in qualitative research.³ Secondary analysis involves using existing data, collected for a prior study, to pursue a question that's distinct from that of the original work.

To analyze the Salion case study data, we used coding techniques from the grounded theory (GT) methodology.⁴ GT coding techniques work best in research into relatively "uncharted land"—for example, integrating SPLE and agile methods. These techniques help inductively discover patterns of interest by careful qualitative data collection and analysis. We applied the three GT coding procedures: *open*, *axial*, and *selective coding*.⁴ These procedures need not occur in a sequence and can be used iteratively throughout the analysis.

Open coding aims to reveal the essential ideas found in the data. The first step decomposes observations into discrete incidents or ideas, labelling each with a name that represents the concepts inherent in the phenomenon. The second step discovers categories by finding related phenomena or common concepts and themes in the accumulated data and grouping them under joint headings. This step identifies data categories and subcategories. We identified 24 categories corresponding to the practices followed and contingencies that affect the integrated process.

Axial coding aims to develop a deeper understanding of the identified categories' relationships. During axial coding, the first task is to connect categories in terms of a sequence of relationships. The second task turns back to the data to validate the relationships.

Selective coding involves integrating the categories to form the initial theoretical framework. First, we develop a story line, a descriptive narrative about the central phenomenon of study. Then, we relate that story line to all the categories found in axial coding. We then validate these relationships and elaborate categories needing further refinement and development.

References

1. P.C. Clements and L.M. Northrop, *Salion, Inc.: A Software Product Line Case Study*, tech. report CMU/SEI-2002-TR-038, Software Eng. Inst., Carnegie Mellon Univ., 2002.
2. R. Buhrdorf, D. Churchett, and C. Krueger, "Salion's Experience with a Reactive Software Product Line Approach," *Software Product-Family Engineering*, LNCS 3014, Springer, 2004, pp. 317-322.
3. J. Heaton, "Secondary Analysis of Qualitative Data," *The Handbook of Social Research Methods*, P. Alasuutari, L. Bickman, and J. Brannen, eds., Sage Publications, 2008.
4. A. Strauss and J. Corbin, *Basics of Qualitative Research: Grounded Theory Procedures and Techniques*, Sage Publications, 1990.

Progressively intertwine domain and application engineering. The choice of a reactive SPLE approach led Salion to balance the extent of product line commonality and variability. A proactive approach requires significant commonality and limited variability that can be predicted up front. However, Salion recognized that considerable variability can't be predicted.

Continuously engage customers to adaptively determine requirements. Initial development focused

Table 1**Mapping blended practices to principles from complexity theory**

Complex adaptive systems (CAS) principles	Implications of CAS principles for process adaptation	Successful practices in the process that integrates software product line engineering and agile methods
Adaptive tension	■ As stakeholder needs evolve, adaptive tension increases. ■ Changes that the system must address if it has to remain effective. ■ Calls for alignment with evolving organizational needs.	■ Progressively intertwine domain and application engineering. ■ Continuously engage customers to adaptively determine requirements. ■ Control scope.
Requisite complexity	■ Same degree of internal complexity to match or destroy the external complexity. ■ Continuous system adaptation to respond to dynamics of the external environment through positive feedback. ■ Inclusion of participants from different backgrounds in the process. ■ Incremental release of new components (or evolutionary development).	■ Establish standard process baselines. ■ Establish a knowledge repository. ■ Use simple but balanced design. ■ Refactor selectively.
Change rate	■ Quick learning action loops. ■ Execute observation-orientation-decision-action loop rapidly. ■ Short cycle development, responsive to requisite changes.	■ Derive products incrementally. ■ Adapt variation mechanisms. ■ Adapt a product line scoping strategy.
Modular design	■ Loose coupling. ■ Minimize interdependencies. ■ Use of a modular architecture.	■ Establish a balanced boundary between domain and application engineering teams. ■ Develop a flexible architecture. ■ Use COTS when feasible. ■ Refactor for reuse.
Positive feedback	■ Development spirals. ■ Prototyping for risk reduction. ■ Rework and go-backs to earlier stages to reduce risks. ■ Specifications—not exhaustive or formal.	■ Provide adequate autonomy. ■ Use risk management practices. ■ Extend the platform to accommodate variability.
Causal intricacy	■ Multidirectional causality. ■ Nonlinear changes. ■ Mutual adaptation. ■ Focus on dynamic interaction of technical, business, and institutional requirements.	■ Manage complex relationships.
Coordination rhythms	■ Circular organizing. ■ Duality of influence between information system (IS) design experts and “user” stakeholders. ■ Balance choices among four domains (business strategy, IS strategy, organizational infrastructure and processes, and IS infrastructure and processes).	■ Facilitate the duality of top-down and bottom-up structures. ■ Align the infrastructure with process needs. ■ Incorporate a mixed organizational structure.

on creating a platform to address one customer’s needs. Salion handled variations for new customers by refactoring the existing platform.

Control scope. Although the flexible architecture and robust development process let Salion handle a wide range of variabilities, the company created mechanisms to balance the conflict between fulfilling all customer requirements and limiting customizations by controlling the scope of variations.

Requisite Complexity

The integrated practices created an adaptable infrastructure, processes, and design principles

that enabled evolution. These practices provided enough complexity to match the external complexity resulting from changing stakeholder needs.

Establish standard process baselines. Salion recognized the importance of establishing a standard process baseline that provided flexibility to handle environmental complexity.

Establish a knowledge repository. On the basis of the project’s complexity and the extent of details involved in managing common and variable requirements, Salion developed a culture among developers to document design decisions, ratio-

nale, and experiences in a structured knowledge repository.

Use simple but balanced design. In handling variable requirements, Salion often chose to develop simpler alternatives using custom code rather than generic components that could lead to more complexity.

Refactor selectively. Instead of refactoring across the board, developers selected the artifacts that would produce the most significant benefits.

Change Rate

Three integrated practices exhibit continuous adaptation and evolution through incremental or evolutionary changes to address changing customer and market needs.

Derive products incrementally. To manage variability effectively, Salion followed a two-phase approach. First, it deployed its fully functional standard platform with common functionality required by the customers. This provided a frame of reference for customers to define their variable requirements and helped Salion customize the platform.

Adapt variation mechanisms. To handle variations quickly, Salion initially developed generic components that could be configured on the basis of variable requirements. However, owing to the complexity involved, the organization soon abandoned this approach and used code customizations instead to handle variability.

Adapt a product line scoping strategy. Variability needed in the software product line evolved as Salion's market maturity increased significantly. Initial development was shaped by wider acceptance of variations to provide whatever the customer required. After a stable product platform emerged, Salion tried to adapt its existing platform to accommodate a limited set of new requirements rather than committing to diverse sets of variations.

Modular Design

Four integrated practices highlight that modularity is essential in both organizational and system design.

Establish a balanced boundary between domain and application engineering teams. Whereas SPLE organizations typically separate their domain and application engineering teams, Salion let all develop-

ers work on any aspect of development. However, when major customizations were required, Salion assigned the developers to separate product development teams.

Develop a flexible architecture. Salion used a layered architecture that was nimble enough to handle changes. The architecture separated runtime debugging from business logic so that developers could focus on the business logic.

Use COTS when feasible. Salion used COTS components whenever the functionality required was relatively less important or when standards already existed. For functionalities without solutions that fit Salion's quality attribute criteria, it developed its own components. The modular nature of design helped Salion achieve such selective COTS use.

Refactor for reuse. Salion used tools that incorporated sophisticated algorithms to extract cloned code that could be reengineered for reuse. The published case study describes these tools in detail.¹²

Positive Feedback

Three integrated practices highlight the ability to progress toward more complex structures through positive feedback.

Provide adequate autonomy. The integrated practices evolved to a more complex structure as the product platform evolved. So, this is a positive feedback process in which inputs about variability result in the integrated process moving toward a more complex structure.

Use risk management practices. Although Salion incorporated seemingly contradictory practices from the rational unified process and XP, it also actively took measures to manage risks. For example, Salion's reactive approach to product line development mitigated its lack of experience in the domain. The organization also held frequent meetings at multiple levels to actively manage risks.

Extend the platform to accommodate variability. Consistent with the reactive product line approach, the initial platform didn't have much capability to handle variations. Instead, developers refactored the platform such that specific modules became more amenable to customization.

Causal Intricacy

One integrated practice highlights the importance of causal intricacy.

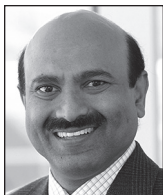
Salion developed a culture among developers to document design decisions, rationale, and experiences in a structured knowledge repository.

About the Authors



Kannan Mohan is an associate professor of computer information systems at Baruch College at the City University of New York. His research interests include managing software product family development, providing traceability support for systems development, knowledge integration, and agile development methodologies. Mohan has a PhD in computer information systems from Georgia State University. Contact him at kannan.mohan@baruch.cuny.edu.

Balasubramaniam Ramesh is a Board of Advisors professor of computer information systems at Georgia State University. His research interests include requirements engineering and traceability, agile software development, and knowledge management. Ramesh has a PhD in information systems from New York University. He's a member of IEEE, the ACM, and the Association for Information Systems. Contact him at bramesh@gsu.edu.



Vijayan Sugumaran is a professor of management information systems at Oakland University and Sogang University. His research interests include software product line engineering, ontologies and the Semantic Web, multiagent systems, and service systems engineering and management. Sugumaran has a PhD in information technology from George Mason University. He's a member of the ACM, the Decision Sciences Institute, and the Association for Information Systems. Contact him at sugumara@oakland.edu.

Manage complex relationships. Salion recognized that critical development decisions involved balancing technological and management needs with platform and product development needs. The organization carefully managed the links between organizational management style, development processes, and technology management, recognizing their complexity, multidirectionality, and nonlinearity.

Coordination Rhythms

Three integrated practices focus on examining and managing dualities in different dimensions and the need for alignment between infrastructure, organizational structure, and changing market needs.

Facilitate the duality of top-down and bottom-up structures. Salion's reactive SPLE approach amplified the complexity of the links between top-down and bottom-up structures, resulting in a dynamic boundary between core-asset and product development teams. At Salion, the separation across core-asset and product development was blurred unless the development team expected complex customizations. Depending on the context, the organization followed a coordinated rhythm in balancing top-down and bottom-up structure.

Align the infrastructure with process needs. Salion deployed the necessary infrastructure to effectively

manage commonality and variations, using tools to identify components employed in each product variant and as customizations to handle variable requirements. Salion also used a tool to detect cloned code that could be refactored. Whenever possible, the organization used simple rather than sophisticated tools. For instance, Salion used an open source configuration management tool rather than sophisticated, proprietary configuration management tools because the former was relatively simple and the team was familiar with it.

Incorporate a mixed organizational structure. One team did initial platform development collectively instead of splitting into separate domain engineering and application engineering teams. After a few cycles, Salion assigned major customizations to separate teams.

Our research shows that software product line development organizations can make SPLE practices more agile by following certain guidelines.

Top management should assess and manage conflicting goals. Using CAS principles to frame integrated practices highlights the need for top management to understand the conflicting goals and processes that evolve continuously to adapt to environmental changes. Top management must examine conflicts in bringing together SPLE and agile practices and foster an environment that balances current organizational needs, past practices, cultural factors, and changing environments.

Top management should also create incentives for adaptability. It should recognize the need to manage the relationships among technological, organizational, and strategic objectives and continuously assess these factors' alignment, recognizing the evolutionary nature of these relationships. In addition, top management should facilitate flexibility to implement appropriate development and project management practices rather than impose organizational mandates to implement one-size-fits-all methods.

Project managers should manage boundaries in teams. They should manage the boundaries between domain and application engineering teams and manage the project scope based on market, organizational, and technological factors. They should encourage practices that promote active collaboration across domain and application engineering.

Project managers should also implement and execute flexible processes. A reactive approach to

SPLE is accompanied by several evolutionary practices that might be inconsistent with a proactive SPLE approach or a more traditional software development approach. So, project managers should recognize the need to afford considerable flexibility in development processes so that they can be effectively adapted to handle changing environmental conditions and stakeholder needs. The organization should implement traditional SPLE processes with flexibility so that developers can react to changing needs in requirements and follow situation-specific practices.

Developers can use most of the practices that have emerged from this study. For instance, practices related to modular system design, variability mechanisms, targeted refactoring, refactoring for reuse, and so on are concrete prescriptions that they can effectively use. As top management and project managers create an environment that empowers developers, developers should carefully maintain the rigor needed for SPLE without sacrificing adaptability.

Although the practices might apply to different contexts that demand integration of agile and SPLE practices, further studies are needed to examine these practices' generalizability. While we've examined the applicability of Salion's reactive SPLE approach, further studies can focus on gaining insights into tailoring proactive SPLE to incorporate agility. ☞

Acknowledgments

The Sogang Business School's World Class University Program (R31-20002), funded by the Korea Research Foundation, partly supported Vijayan Sugumaran's work.

References

1. P. Clements and L. Northrop, *Software Product Lines: Practices and Patterns*, Addison-Wesley, 2002.
2. P. Abrahamsson et al., "New Directions on Agile Methods: A Comparative Analysis," *Proc. 25th Int'l Conf. Software Eng. (ICSE 03)*, IEEE CS Press, 2003, pp. 244–254.
3. G.K. Hanssen and T.E. Faegri, "Process Fusion: An Industrial Case Study on Agile Software Product Line Engineering," *J. Systems and Software*, vol. 81, no. 6, 2008, pp. 843–854.
4. S. Black et al., "Formal versus Agile: Survival of the Fittest," *IEEE Computer*, vol. 42, no. 9, 2009, pp. 37–45.
5. K. Tian and K. Cooper, "Agile and Software Product Line Methods: Are They So Different?" Workshop Agile Product Line Eng. (APLE 06), 2006; www.lsi.upc.edu/events/aple/TianCooper.pdf.
6. K. Cooper and X. Franch, "Editorial," *J. Systems and Software*, vol. 81, no. 6, 2008, pp. 841–842.
7. J.H. Holland, *Hidden Order: How Adaptation Builds Complexity*, Basic Books, 1996.
8. R. Vidgen and X. Wang, "Coevolving Systems and the Organization of Agile Software Development," *Information Systems Research*, vol. 20, no. 3, 2009, pp. 355–376.
9. P. Meso and R. Jain, "Agile Software Development: Adaptive Systems Principles and Best Practices," *Information Systems Management*, vol. 23, no. 3, 2006, pp. 19–30.
10. X. Wang and K. Conboy, "Understanding Agility in Software Development through a Complex Adaptive Systems Perspective," presented at 17th European Conf. Information Systems (ECIS 09), 2009; www.ecis2009.it/papers/ecis2009-0561.pdf.
11. H. Benbya and B. McKelvey, "Toward a Complexity Theory of Information Systems Development," *Information Technology & People*, vol. 19, no. 1, 2006, pp. 12–34.
12. P.C. Clements and L.M. Northrop, *Salion, Inc.: A Software Product Line Case Study*, tech. report CMU/SEI-2002-TR-038, Software Eng. Inst., Carnegie Mellon Univ., 2002.



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.

Silver Bullet Security Podcast

In-depth interviews with security gurus. Hosted by Gary McGraw.

www.computer.org/security/podcasts

Sponsored by SECURITY & PRIVACY digital